

Kickoffs-MEAN-Book_My_Movie

Explained Problem Statement with Solution Notes

What this mock hands-on is testing

This task checks whether you can read an exact requirement, keep field names unchanged, write a simple Angular UI, connect it to a Node + Mongo backend, validate input strictly, and return the exact alert messages and HTTP status codes expected by automated tests.

1. Original-style problem statement

- Build an application that is used to book tickets for movies and search for and view the details of the movie.
- The front end must show all movies from the backend, allow searching by movie name, and allow ticket booking for a selected movie.
- The backend must expose one GET API to fetch movie data and one PATCH API to decrement available seats when booking is successful.

Front-end requirements (Angular)

- Display all movies and their details in the AppComponent.
- Each movie should have a Book button.
- Movie details should come from the backend.
- When the user clicks Book, the app should collect the ticket count from the user.
- Use a template reference variable to read the input value.
- If booking is successful, show the alert message: "Tickets booked successfully".
- If the input is a string, special character, blank space, zero, or otherwise invalid, show the alert message: "Input field should be valid".
- If available seats are zero, show the alert message: "The tickets are sold out".
- If the entered ticket count is greater than available seats, show the alert message: "Sorry, we only have \${available_tickets} tickets available. Your request for \${user_entered_ticketCount} tickets cannot be fulfilled."
- Provide a search input that filters the list by movie name.

Back-end requirements (NodeJS + MongoDB)

- Database name: moviebooking
- Collection/model file: movie.js inside src/models

Schema fields

Field Name	Type	Required	Meaning
_id	ObjectID	Auto-generated	Unique identifier for each movie
movie_name	String	TRUE	Movie title shown in the UI and used for search
show_date	Date	TRUE	Date of the movie show
theatre	String	TRUE	Theatre name or screen location
available_seats	Number	TRUE	Remaining seats that can still be booked

Routers / endpoints

Method	Route	Expected action	Status code rule
GET	/movie	Fetch all movie records from MongoDB	200 on success, 400 on failure
PATCH	/movie/:id	Decrease available_seats using the ticket count sent in the request body	200 on successful decrement, 400 when validation fails or update is not done

2. What the question really means

This is not a design-heavy project. The main goal is to satisfy automated tests.

The app is small, but the instructions are exact. Tests usually fail because of tiny mismatches such as wrong field names, wrong body keys, wrong success message, or missing validation order.

You should assume the starter project is partially built already. Your job is to complete missing code, not redesign the whole app.

How to read this kind of Fresco/TCS problem

- Keep schema field names exactly as provided: movie_name and available_seats, not camelCase alternatives.
- Use the exact request body key mentioned in the statement: { "tickets": number }.
- Treat front-end alert messages and back-end API messages as separate expectations when the wording differs.
- Do not change route paths. If the statement says /movie, use /movie.
- Validate before calling the backend whenever the statement asks for UI-side validation.

3. Solution overview

Part	What the solution does
Mongo model	Defines movie_name, show_date, theatre, and available_seats exactly as in the problem statement.
GET /movie	Reads all movie records and returns them with status 200.
PATCH /movie/:id	Receives { tickets }, validates it, checks availability, decrements available_seats, and returns status 200 with a success message.
Angular service	Calls GET /movie and PATCH /movie/:id so the component stays clean.
Angular component	Loads movies, filters them by search text, validates the entered ticket count, shows exact alerts, and refreshes the movie list after successful booking.

Why the front-end logic is written in that order

1. First, check whether available_seats is 0. If there are no seats, there is no point validating a booking count further.
2. Second, validate the input format. This catches alphabets, special characters, empty strings, and blank spaces before an API call is made.
3. Third, convert the value to a number and reject zero or negative values.
4. Fourth, compare the requested count with available_seats. If the request exceeds stock, show the exact “Sorry...” alert.
5. Only after all validation passes should the app call the PATCH endpoint.

Why the back-end logic is written that way

- The backend still validates tickets even if the frontend already does validation. This protects the API from bad requests and also helps tests that hit the endpoint directly.
- The document is first fetched by id so the code can compare the requested ticket count with available_seats before updating.
- The route returns 400 for invalid request scenarios because the statement explicitly says to return 400 when validation fails or the update is not done.
- The decrement operation updates only available_seats, which is the business field the question cares about.

4. Validation rules mapped to expected alerts

Condition	Front-end action	Expected message
Input is empty / blank / alphabet / special character	Do not call backend	Input field should be valid
available_seats is 0	Do not call backend	The tickets are sold out

Condition	Front-end action	Expected message
Entered ticket count is greater than available_seats	Do not call backend	Sorry, we only have \${available_tickets} tickets available. Your request for \${user_entered_ticketCount} tickets cannot be fulfilled.
Valid ticket count within limit	Call PATCH /movie/:id with { tickets }	Tickets booked successfully (front end)

5. Hidden test case traps to watch for

- Using movieName or availableSeats instead of movie_name and available_seats.
- Sending the PATCH body as { count: 2 } or { seats: 2 } instead of { tickets: 2 }.
- Showing “Ticket booked successfully” in the UI instead of the required “Tickets booked successfully”.
- Calling the backend even when the user entered invalid input.
- Forgetting to reload the movie list after a successful booking, so the UI still shows old seat counts.
- Using route paths such as /movies or /book instead of /movie and /movie/:id.

6. Short implementation checklist

Task	Done when...
Model created	Schema matches the statement exactly.
GET route completed	Calling /movie returns all data with 200.
PATCH route completed	Booking decrements available_seats and returns 200 on success.
Search implemented	Typing movie text filters the displayed list.
Booking validation completed	All three alert cases behave exactly as required.
Refresh completed	After booking, seats shown in the UI are updated from the backend.

Use this document as a reading guide before solving the actual hands-on.